



Chapter 4 – Requirements Engineering

Topics covered



- ✧ **The software requirements document**
- ✧ **Functional and non-functional requirements**
- ✧ Requirements specification
- ✧ Requirements engineering processes
- ✧ Requirements elicitation and analysis
- ✧ Requirements validation
- ✧ Requirements management

Requirements engineering



- ✧ The **system requirements** are the descriptions of the system services and constraints that are generated during the requirements engineering process.
- ✧ **Requirements engineering (RE)**: is the process of finding out, analyzing, documenting, and checking these services and constraints.
 - The process of establishing the services that a customer requires from a system and the **constraints** under which it operates and is developed.

What is a requirement?



- ✧ It may range from a **high-level abstract** statement of a service or of a **system constraint** to a detailed **mathematical functional specification**.
- ✧ This is inevitable as requirements may serve a dual function (Davis 1993):
 - May be the basis for a **bid for a contract** – must be open to interpretation;
 - May be the basis for the **contract itself** - must be defined in detail;

Types of requirement



✧ User requirements

- Statements in natural language plus diagrams of the services the system provides and its operational constraints.
 - Should be written for customers.

✧ System requirements

- A structured document setting out detailed descriptions of the system's functions, services, and operational constraints.

User and system requirements



User requirements definition

- 1.** The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2** The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4** If drugs are available in different dose units (e.g. 10mg, 20mg, etc) separate reports shall be created for each dose unit.
- 1.5** Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

System stakeholders



✧ Any **person or organization that is affected by the system** in some way and has a legitimate interest

✧ Stakeholder types:

- End users
- System managers
- System owners
- External stakeholders

CPUT stakeholders



✧ ?

✧ Mentcare system?

Stakeholders in the Mentcare system



- ✧ **Patients** whose information is recorded in the system.
- ✧ **Doctors** who are responsible for assessing and treating patients.
- ✧ **Nurses** who coordinate the consultations with doctors and administer some treatments.
- ✧ **Medical receptionists** who manage patients' appointments.
- ✧ **IT staff** who are responsible for installing and maintaining the system.

Stakeholders in the Mentcare system



- ✧ A **medical ethics manager** who must ensure that the system meets current ethical guidelines for patient care.
- ✧ **Health care managers** who obtain management information from the system.
- ✧ **Medical records staff** who are responsible for ensuring that system information can be maintained and preserved.

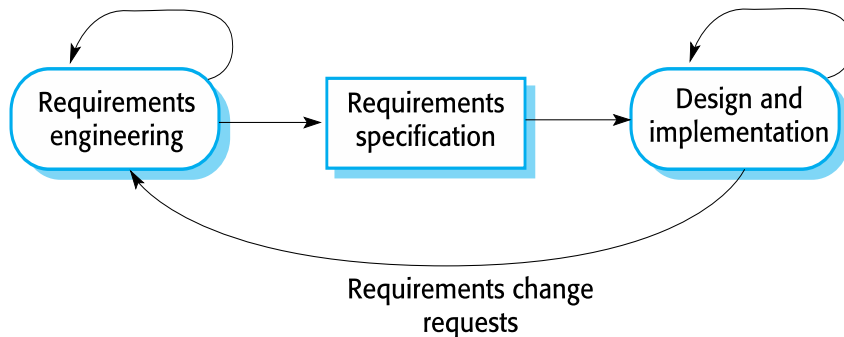
Agile methods and requirements



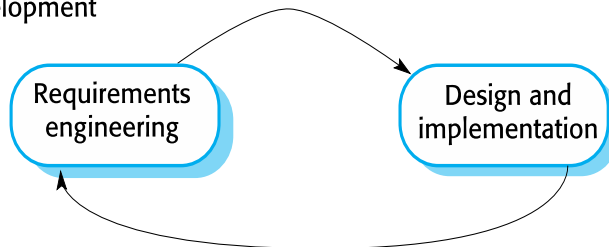
- Agile manifesto!

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

Plan-based development



Agile development



Agile methods and requirements



- ✧ Many **agile methods** argue that **producing detailed system requirements is a waste of time**, as requirements change so quickly.



Functional and non-functional requirements

Functional and non-functional requirements



✧ Functional user requirements:

✧ High-level statements of what the system should do.

✧ What does the system do?

- What type of **services** the system should provide in **detail**
- How the system should **react** to particular inputs
- How the system should **behave** in particular situations.
- **What the system should not do.**

Functional and non-functional requirements



✧ Non-functional requirements - How does the system perform?

- Constraints on the services or functions offered by the system, such as **timing constraints**, constraints on the **development process, standards**, etc.
- Often apply to the system as a whole rather than individual features or services.

Requirements imprecision



✧ Functional requirements should be **precisely stated**.

✧ **Ambiguous** requirements should be avoided

- Interpretation may affect **developers** and **users**.

✧ Consider the term '**search**' in requirement 1

- **User intention** – search for a patient name across all appointments in all clinics;
- **Developer interpretation** – search for a patient name in an individual clinic.

Requirements completeness and consistency



- ✧ In principle, requirements should be **both complete and consistent**.
- ✧ **Complete**
 - They should include **descriptions of all facilities required**.
- ✧ **Consistent**
 - There should be **no conflicts or contradictions** in the descriptions of the system facilities.

Non-functional requirements



✧ These **define system properties and constraints** e.g.

- Reliability
- Response time
- Memory used
- Storage requirements.

✧ **Constraints on the system implementation:**

- I/O device capability,
- data representations used, etc.

Non-functional requirements



- ✧ **Process requirements** may also be specified:
 - mandating a particular IDE,
 - programming language or development method.

- ✧ **Non-functional requirements may be more critical than functional requirements.**
 - **If these are not met, the system may be useless!**
 - example, **if an aircraft system does not meet its reliability requirements**, it will not be certified as safe for operation

Non-functional requirements implementation



- ✧ Non-functional requirements **may affect the overall architecture of a system rather than the individual components.**
- ✧ A single non-functional requirement, such as a **security requirement**, **may generate a number of related functional requirements** that define system services that are required.

Non-functional classifications



✧ Product requirements

- Requirements which specify that the delivered **product must behave in a particular way** e.g. **execution speed, reliability, acceptable failure rate**, etc.

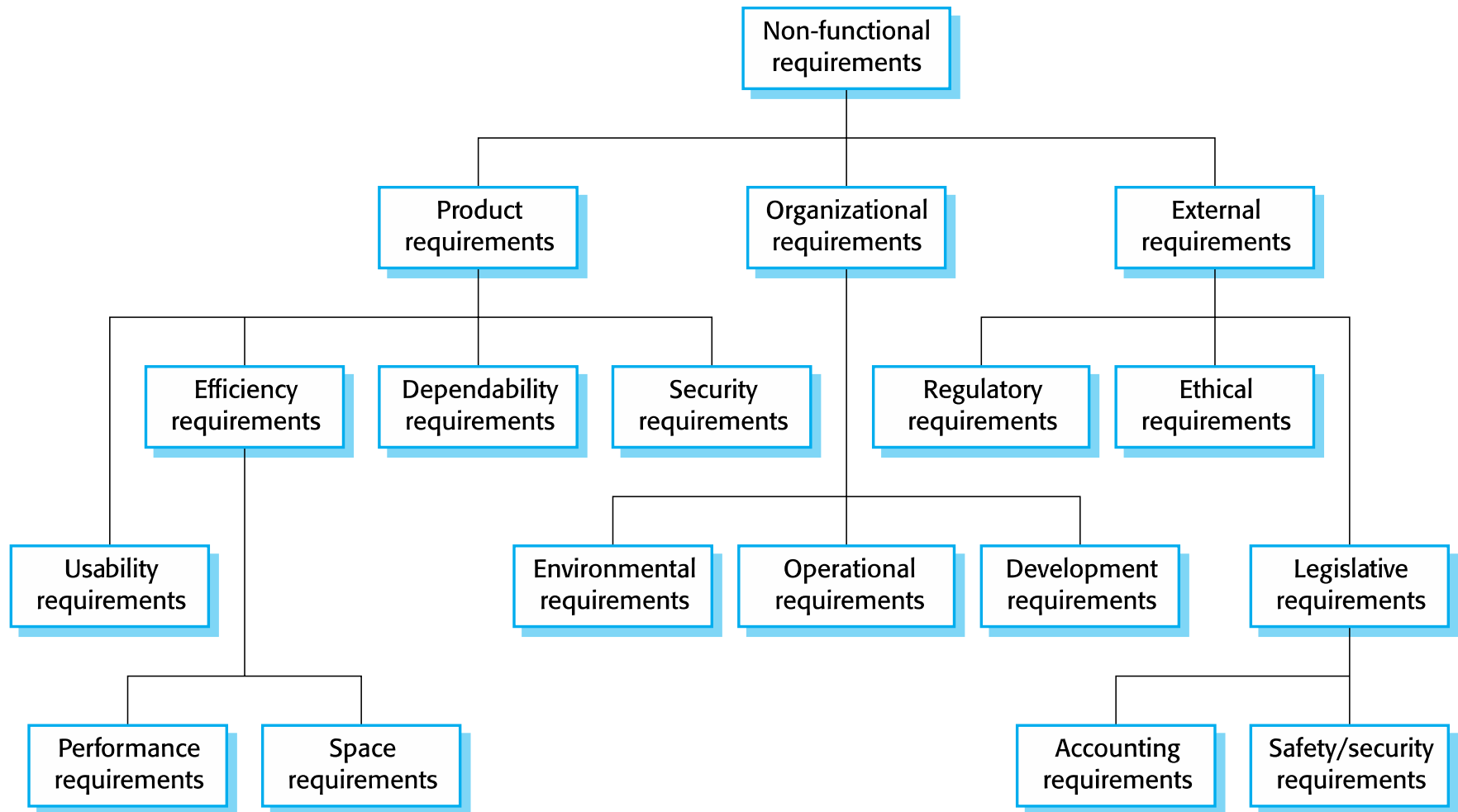
✧ Organisational requirements

- Requirements which are a **consequence of organisational policies and procedures** e.g. **process standards used, implementation requirements**, etc.

✧ External requirements

- Requirements which arise from factors which are **external to the system and its development process** e.g. **interoperability requirements, legislative requirements, ethical requirements**, etc.

Types of nonfunctional requirement



Examples of nonfunctional requirements in the Mentcare system



Product requirement

The Mentcare system shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). **Downtime within normal working hours shall not exceed five seconds in any one day.**

Organizational requirement

Users of the Mentcare system shall **authenticate themselves using their health authority identity card.**

External requirement

The system shall **implement patient privacy provisions** as set out in HStan-03-2006-priv.

Goals and requirements



✧ Goal

✧ A general intention of the user:

- ease of use,
- the ability of the system to recover from failure, or rapid user response.

✧ Goals are **helpful to developers** as they convey the intentions of the system users.

Usability requirements



- ✧ The system should be easy to use, and errors should be minimized. (**Goal**)

- ✧ Verifiable non-functional requirement (**quantitatively**)
 - A statement using some measure that can be objectively tested.

- ✧ **Medical staff** shall be able to use all the system functions after **four hours** of training.

- ✧ After this training, the average number of errors made by experienced users shall not exceed **two per hour** of system use. (Testable non-functional requirement)

Other nonfunctional requirement requirements?



✧ ?

Metrics for specifying nonfunctional requirements



Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems



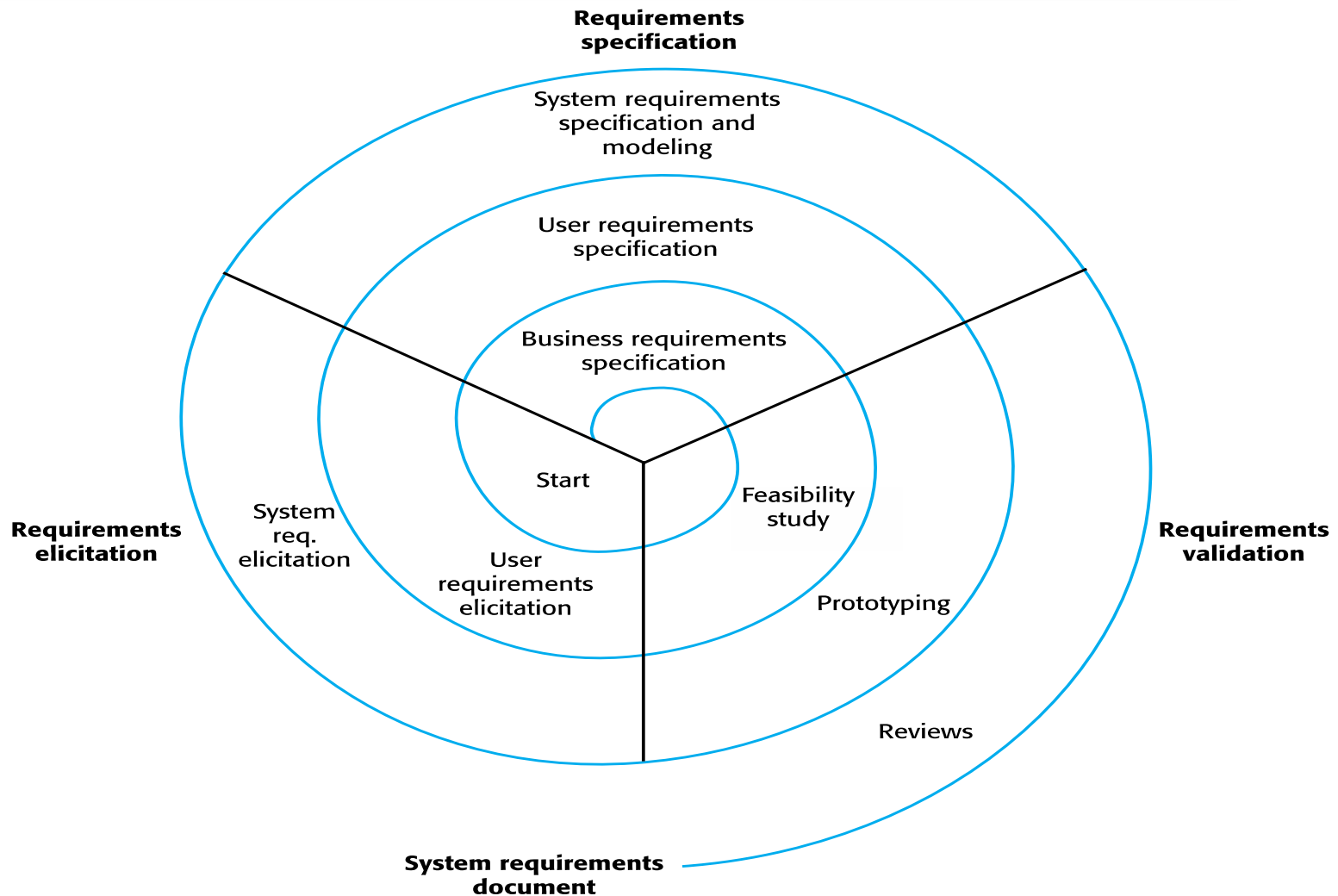
Requirements engineering processes

Requirements engineering processes



- ✧ The **processes** used for RE vary widely depending:
- ✧ The **application domain**,
- ✧ The **people involved**, and
- ✧ The **organisation** developing the requirements.
- ✧ Generic activities common to all processes
 - Requirements **elicitation**;
 - Requirements **analysis**;
 - Requirements **validation**;
 - Requirements **management**.
- ✧ In practice, RE is an **iterative activity** in which these processes are **interleaved**.

A spiral view of the requirements engineering process





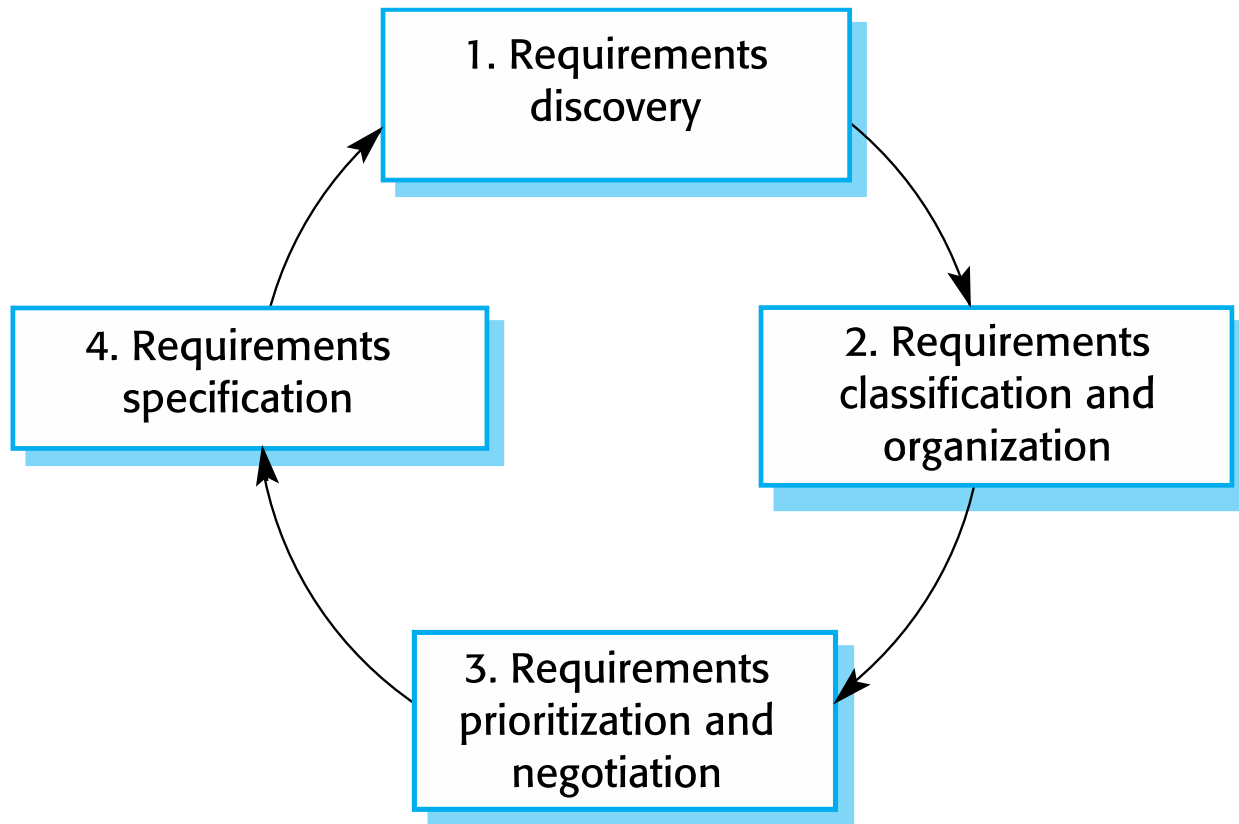
Requirements elicitation

Requirements elicitation and analysis



- ✧ **Requirements elicitation.**
- ✧ Software engineers work with system stakeholders :
- ✧ Stages include:
 - Requirements **discovery** and **Understanding**
 - Requirements **classification** and **organization**,
 - Requirements **prioritization** and **negotiation**,
 - Requirements **specification**.

The requirements elicitation and analysis process



Process activities



✧ Requirements discovery and Understanding

- Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.

✧ Requirements classification and organisation

- Groups related requirements and
- organises them into coherent clusters.

✧ Prioritisation and negotiation

- Prioritising requirements and resolving requirements conflicts.

✧ Requirements specification

- Requirements are documented and input into the next round of the spiral.

Requirements elicitation techniques:

Interviewing



- ✧ **Formal or informal interviews** with stakeholders are part of most RE processes.
- ✧ **Types of interview**
 - **Closed interviews** based on a pre-determined list of questions
 - **Open interviews** where various issues are explored with stakeholders.
- ✧ **Effective interviewing**
 - Be **open-minded**, avoid pre-conceived ideas about the requirements, and be willing to **listen to stakeholders**.
 - Prompt the interviewee to get discussions going using a **springboard question**, a **requirements proposal**, or by working together on a **prototype system**.

Interviews in practice



- ✧ Normally, a mix of closed and open-ended interviewing.
- ✧ Interviews are good for:
 - Getting an overall understanding of what stakeholders
 - How stakeholders interact with the system.
- ✧ Interviewers need to be open-minded without pre-conceived ideas of what the system should do
- ✧ You need to prompt the user to talk about the system by suggesting requirements rather than simply asking them what they want.

Question!



✧ **Do you think that using both open and closed interviews is sufficient to capture all stakeholder requirements?**

Problems with interviews



- ✧ Application specialists may **use language to describe their work that isn't easy for the requirements engineer** to understand.
- ✧ Interviews are **not good for understanding domain requirements**
 - Requirements engineers cannot understand **specific domain terminology**;
 - **Some domain knowledge is so familiar** that people find it hard to articulate or think that it isn't worth articulating.

Ethnography



- ✧ A social scientist spends a considerable time **observing and analysing how people actually work.**
- ✧ People do **not have to explain or articulate their work.**
 - **Social** and **organisational factors** of importance may be observed.
- ✧ **Ethnographic** studies have **shown that work is usually richer and more complex** than suggested by simple system models.

Scope of ethnography

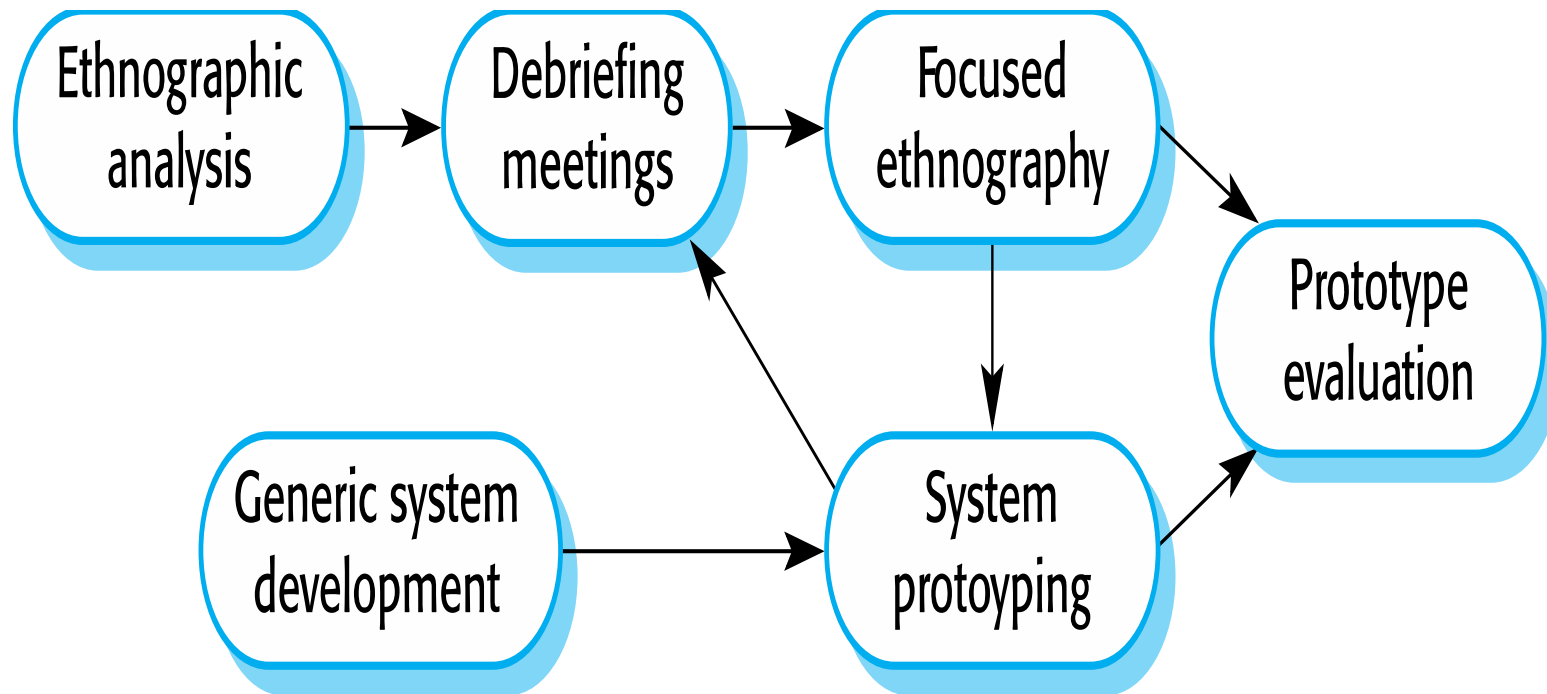


✧ Requirements that are derived from:

- The **way that people actually work** rather than the way in which process definitions suggest that they ought to work.
- **Cooperation and awareness of other people's activities.**

✧ **Ethnography** is **effective for understanding existing processes, but cannot identify new features** that should be added to a system.

Ethnography and prototyping for requirements analysis



Focused ethnography



- ✧ Developed in a project **studying the air traffic control process**
- ✧ Combines **ethnography with prototyping**
 - **Prototype** development **results in unanswered questions** that focus the ethnographic analysis.
- ✧ The **problem with ethnography is that it studies existing practices, which may have some historical basis that is no longer relevant.**

Question!



- ✧ Do you think that combining **ethnographic research with prototyping** is sufficient to capture all stakeholder requirements and develop a competitive user product?
- ✧ In addition, what other challenges might limit the effectiveness of ethnography?

Clue!



✧ Innovation and broader organizational domain!

✧ Nokia vs. Apple!

Question/LTD!



- ✧ You decide to use **ethnography** by spending a week observing the nurses on the ward. What specific things might you observe about their workflow that they would never think to tell you in an interview?



Requirements specification

Requirements specification



- ✧ The process of writing down the user and system requirements in a requirements document.
 - **User requirements** - understandable by end-users and customers who do not have a technical background.
 - **System requirements** - more detailed requirements, and may include more technical information.

- ✧ The requirements may be **part of a contract.**
 - Important that these are as complete as possible.

Ways of writing a system requirements specification



Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template . Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language similar to a programming language , but with more abstract features to specify requirements by defining an operational model of the system. This approach is now rarely used, although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by textual annotations, are used to define the system's functional requirements ; UML use case and sequence diagrams are commonly used .
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets . Although these unambiguous specifications can reduce ambiguity in a requirements document, most customers don't understand formal specifications. They cannot check that it represents what they want, and are reluctant to accept it as a system contract

Requirements and design



- ✧ In principle,
 - **Requirements** should state what the system should do
 - **Design** should describe how it does it.
- ✧ In practice, **requirements and design are inseparable**
 - A **system architecture** may be designed to structure the requirements;
 - The **system may interoperate with other systems** that generate design requirements;
 - The use of a **specific architecture to satisfy non-functional requirements** may be a domain requirement.

Natural language specification



- ✧ Requirements are written as natural language sentences **supplemented by diagrams and tables**.
- ✧ Used for writing requirements because it is **expressive**, **intuitive**, and **universal**.
 - This means that the requirements can be **understood by users** and **customers**.

Guidelines for writing requirements



- ✧ Invent a standard format and use it for all requirements.
- ✧ Use language in a consistent way.
- ✧ Use text highlighting to identify key parts of the requirement.
- ✧ Avoid the use of computer jargon.
- ✧ Include an explanation (rationale) of why a requirement is necessary.

Problems with natural language



✧ **Lack of clarity**

- Precision is difficult!

✧ **Requirements confusion**

- Functional and non-functional requirements tend to be mixed up.

✧ **Requirements amalgamation**

- Several different requirements may be expressed together.

Form-based specifications



- ✧ Definition of the **function or entity**.
- ✧ Description of **inputs** and where they come from.
- ✧ Description of **outputs** and where they go.
- ✧ Description of **computation** and other entities used.
- ✧ Description of the **action to be taken**.
- ✧ **Pre and post conditions** (if appropriate).
- ✧ The **side effects** (if any) of the function.

Tabular specification



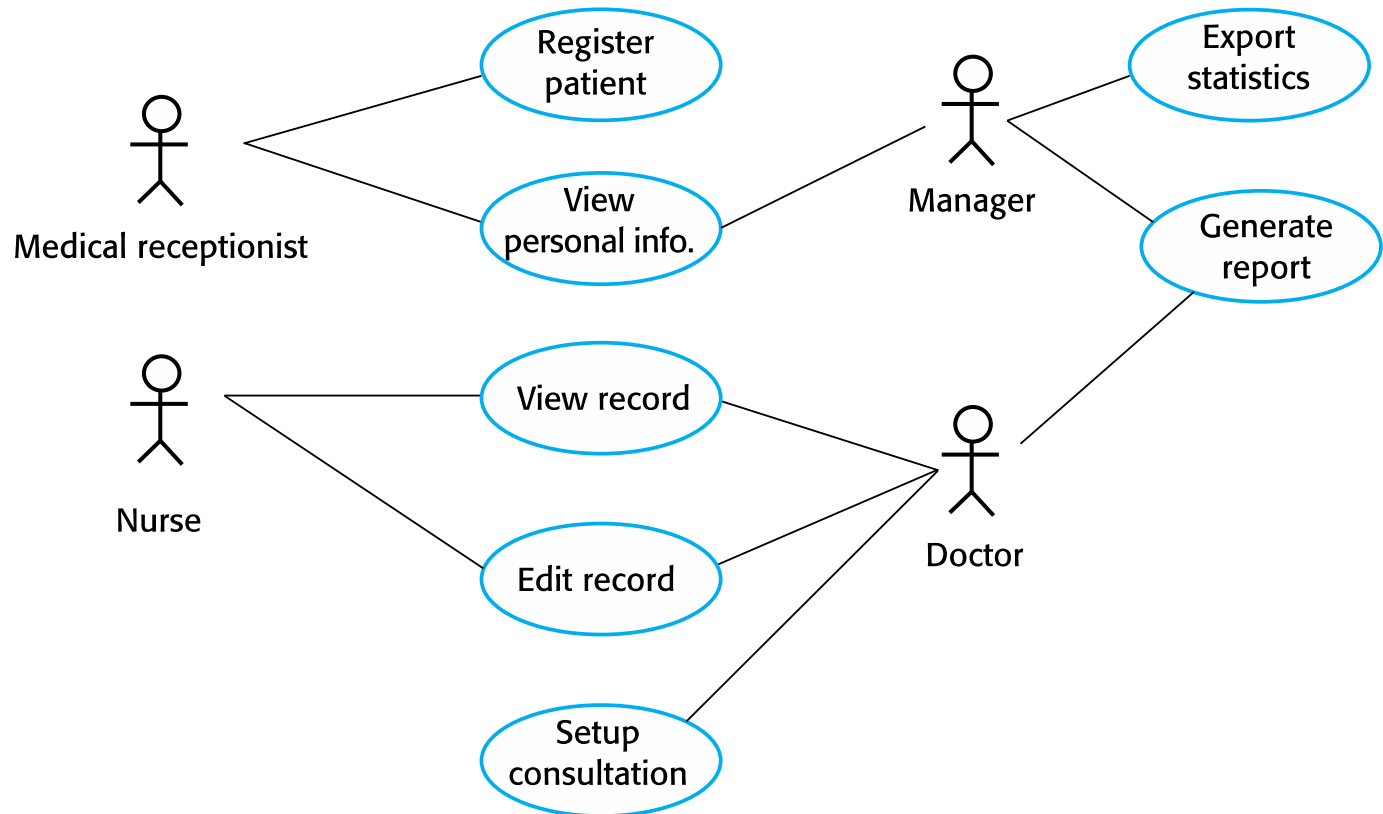
- ✧ Used to supplement natural language.
- ✧ Useful to define a number of possible alternative courses of action.
- ✧ **Example: the insulin pump system**
 - Bases its computations on the rate of change of the blood sugar level
 - Tabular specification explains how to calculate the insulin requirement for different scenarios.

Use cases



- ✧ **Use-cases** are a kind of scenario that are included in the **Unified Modeling Language (UML)**
- ✧ **Use cases** identify the actors in an interaction and describe the interaction itself.
- ✧ **A set of use cases** - describe all possible interactions with the system.
- ✧ **UML sequence diagrams** - used to add detail to use cases by showing the sequence of event processing in the system.

Use cases for the Mentcare system

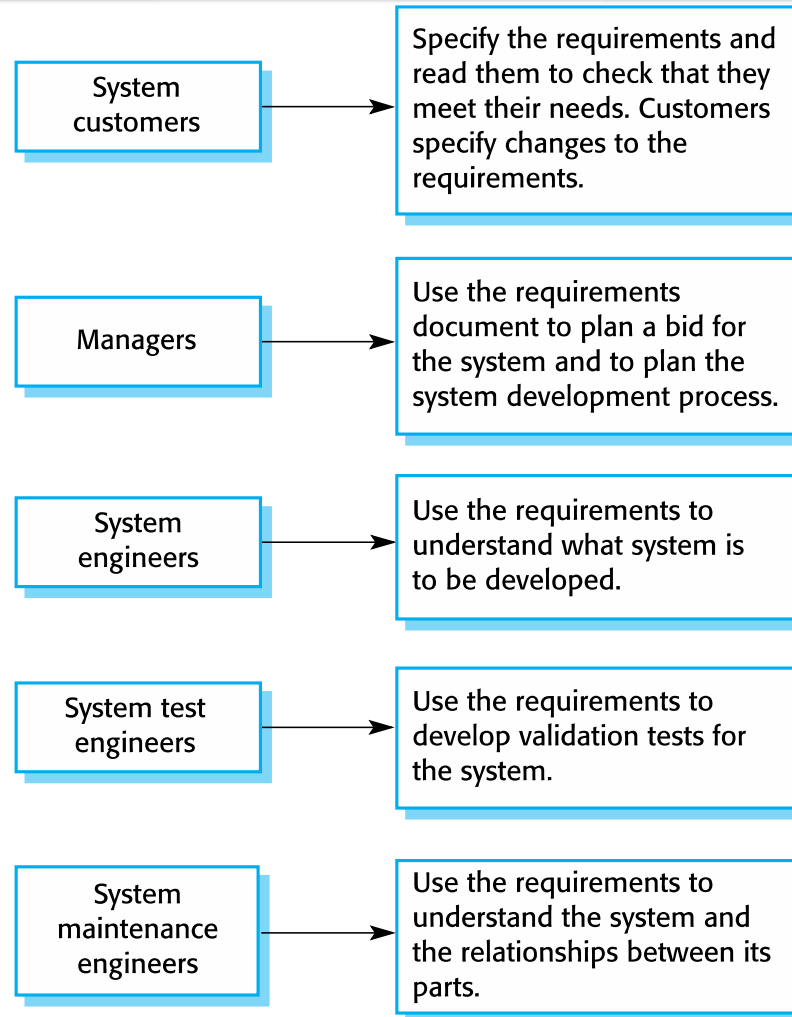


The software requirements document



- ✧ The **software requirements document** - official statement of what is required of the system developers.
 - Include both a definition of user requirements and a specification of the system requirements.
 - It is **NOT** a design document but set of WHAT the system should do rather than HOW it should do it.

Users of a requirements document



The structure of a requirements document



Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user . The non-functional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture , showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

The structure of a requirements document



Chapter	Description
System requirements specification	This should describe the functional and non-functional requirements in more detail . If necessary, further details may also be added to the non-functional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs , and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information related to the application being developed, such as hardware and database descriptions . Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indices to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.



Requirements validation

Requirements validation



- ✧ Concerned with demonstrating that the requirements **define the system that the customer really wants.**
 - Requirements error costs are high, so validation is very important
 - Fixing a requirements error after delivery may cost up to **100 times** the cost of fixing an implementation error.

Requirements checking



- ✧ **Validity:** Does the system provide the functions that best support the customer's needs?
- ✧ **Consistency.** Are there any requirements conflicts?
- ✧ **Completeness.** Are all functions required by the customer included?
- ✧ **Realism.** Can the requirements be implemented, given the available budget and technology
- ✧ **Verifiability.** Can the requirements be checked?

Requirements validation techniques



✧ Requirements reviews

- Systematic **manual analysis** of the requirements.

✧ Prototyping

- Using an **executable model of the system to check requirements**.

✧ Test-case generation

- **Developing tests** for requirements to check testability.

Requirements reviews



- ✧ **Regular reviews** should be held while the requirements definition is being formulated.
 - Both **client** and **contractor** staff should be involved in reviews.
 - Reviews may be **formal** or **informal**.
 - **Good communication** among developers, customers, and users can help resolve problems early.

Review checks



✧ Verifiability

- Is the requirement **realistically testable**?

✧ Comprehensibility

- Is the requirement **properly understood**?

✧ Traceability

- Is the origin of the requirement **clearly stated**?

✧ Adaptability

- Can the requirement be **changed without a large impact** on other requirements?

Question!



✧ In your own words, why is it dangerous to start coding before the requirements are validated?



- ✧ You are halfway through building a new e-commerce website. The client informs you that they have just acquired a smaller company, and the new system must now integrate with the acquired company's legacy inventory system. This was not in the original requirements.
- ✧ Q1: According to the change management process, what is the very first thing you should do? (Problem analysis).
- ✧ Q2: What factors would you analyze to determine the cost and impact of this change?
- ✧ Q3: Who should be involved in the final decision to accept or reject this change?